

8 Microservices Mistakes Killing Your Apps



Synchronous Requests Between Services

❌ Problem:

Calling Service $A \rightarrow B \rightarrow C \rightarrow D$ in a chain for every request.

✅ Solution:

Use async messaging (RabbitMQ, Kafka) for non-critical flows. Reserve HTTP calls for truly real-time needs.

One slow service freezes the whole request. Your total timeout becomes the sum of every service in the chain.



02 Shared Database Between Services

✗ Problem:

Multiple services reading and writing the same database tables.

✓ Solution:

Give each service its own database. Communicate through APIs or events, not shared tables.

This is the #1 sign you have a distributed monolith, not microservices. You lose independent deployability without even realizing it.



No API Gateway

❌ Problem:

Clients calling each service directly by URL.

✅ Solution:

Add an API Gateway (YARP, Ocelot, AWS API Gateway) as the single entry point.

Without a gateway, every service handles auth, rate limiting, and SSL on its own. You repeat the same code everywhere.



Missing Circuit Breaker

✗ Problem:

No fallback when a downstream service goes down.

✓ Solution:

Use Polly in .NET to add circuit breakers, retries, and timeouts on all outbound calls.

One unhealthy service cascades into a full system outage. A circuit breaker stops the failure from spreading to the rest of your system.



No Distributed Tracing

✗ Problem:

Each service logs in isolation with no way to connect the dots.

✓ Solution:

Add OpenTelemetry and send traces to Jaeger, Zipkin, or your cloud provider.

When a bug spans 5 services, you need one trace ID to follow the full request path. Without it, debugging is pure guesswork.



Ignoring Eventual Consistency

✗ Problem:

Expecting immediate data consistency across services just like in a monolith.

✓ Solution:

Design for eventual consistency. Use the Outbox Pattern to safely publish events after database writes.

Microservices are distributed by nature. Forcing strong consistency creates hidden data conflicts and broken business logic under load.



Hardcoded Service URLs

✗ Problem:

Services calling each other with hardcoded IPs or URLs in config files.

✓ Solution:

Use service discovery (Consul, Kubernetes DNS, .NET Aspire) so services find each other dynamically.

One redeploy or IP change breaks everything. Service discovery removes this fragile hidden dependency entirely.



08 No Idempotent Message Consumers

✗ Problem:

Processing messages from a queue with no duplicate checks.

✓ Solution:

Store a processed message ID before handling. Skip duplicates on retry.

Message brokers guarantee at-least-once delivery. Without idempotency, retries silently create duplicate orders, payments, or database records.



Repost & Follow [Anton](#) for more content like this

Want to improve
.NET Skills?

Subscribe to my
[AntonDevTips](#)
newsletter

